

Personal Library — Camunda BPMN 2.0 Specification

Automated Document Ingestion, Parallel Dual AI Summarization & Qdrant Vector RAG

PROCESS IDENTIFIER	ENGINE COMPATIBILITY	LICENSE & STANDARD
Process_DocumentIngestionRAG	Camunda 7 (Spring Boot) & Camunda 8 (Zeebe)	GNU AGPLv3 · BPMN 2.0 with BPMNDI

Executive Summary & Pipeline Architecture

The Personal Library Ingestion Engine orchestrates the lifecycle of research documents from initial upload to vector retrieval. Implemented as an executable BPMN 2.0 process, it bridges the SAP Fiori UI, Spring AI orchestration, Ollama local inference models, and the Qdrant vector database into a fault-tolerant workflow with built-in human fallback.

Key Architectural Tenets:

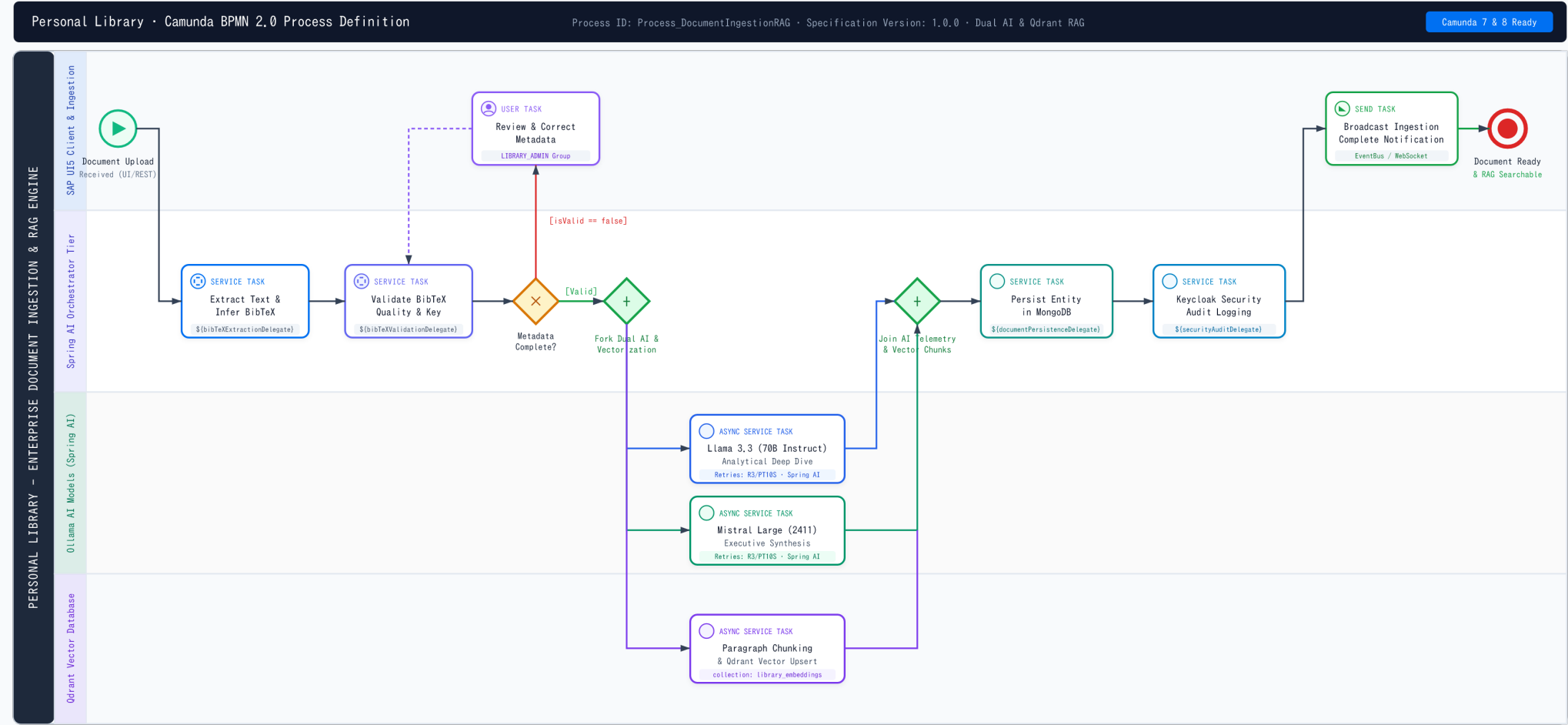
- 4 Horizontal Swimlanes: Client Ingestion (UI5/REST), Spring AI Orchestrator, Ollama Dual Models, and Qdrant Vector Store.
- Parallel Asynchronous Execution: Forks dual-model summarization across Ollama Llama 3.3 (70B Instruct) for analytical breakdown and Mistral Large (2411) for executive synthesis concurrently with paragraph semantic chunking.
- Human-in-the-Loop Quality Gate: An Exclusive XOR Gateway routes incomplete or malformed bibliographic extractions to a Librarian Review User Task, preventing corrupted metadata from reaching downstream indexing.
- Enterprise Governance: Atomic persistence in MongoDB coupled with tenant-scoped security audit logging in Keycloak and real-time push notifications over the SAP UI5 EventBus.

Engine Support:

- Camunda 7: JavaDelegate expression binding via Spring Boot Starter with asynchronous continuation.
- Camunda 8: Non-blocking Zeebe JobWorkers with automated backoff retry policies (R3/PT10S).

Camunda BPMN 2.0 Process Diagram

Complete Visual Workflow with Horizontal Swimlanes & Orthogonal Waypoints



Process Task Catalog & Execution Matrix

Technical specification of process activities, delegates, and execution modes

Activity / Task Name	Camunda 7 Delegate	Zeebe Task Type (C8)	Swimlane	Execution	Retries
Extract Text & BibTeX	<code>\${bibTeXExtractionDelegate}</code>	extract-bibtex-task	Spring AI Orchestrator	Synchronous	3
Validate BibTeX Quality	<code>\${bibTeXValidationDelegate}</code>	validate-bibtex-task	Spring AI Orchestrator	Synchronous	None
Review & Correct Metadata	User Task (Librarian)	User Task (Human)	SAP UI5 Client	Manual Review	N/A
Fork Dual AI & Vectorization	Parallel AND Gateway	Parallel AND Gateway	Spring AI Orchestrator	Fork Split	N/A
Llama 3.3 (70B) Summary	<code>\${llamaSummarizationDelegate}</code>	llama-summary-task	Ollama AI Models	<code>asyncBefore=true</code>	R3/PT10S
Mistral Large (2411) Summary	<code>\${mistralSummarizationDelegate}</code>	mistral-summary-task	Ollama AI Models	<code>asyncBefore=true</code>	R3/PT10S
Chunk Text & Upsert Vectors	<code>\${vectorRagDelegate}</code>	qdrant-vectorization-task	Qdrant Vector DB	<code>asyncBefore=true</code>	R3/PT10S
Join AI & Vector Telemetry	Parallel AND Join	Parallel AND Join	Spring AI Orchestrator	Join Barrier	N/A
Persist Entity in MongoDB	<code>\${documentPersistenceDelegate}</code>	persist-document-task	Spring AI Orchestrator	Synchronous	3
Keycloak Security Audit Log	<code>\${securityAuditDelegate}</code>	keycloak-audit-task	Spring AI Orchestrator	Synchronous	3
Broadcast Ingestion Event	<code>\${eventBusNotificationDelegate}</code>	notify-client-task	SAP UI5 Client	Send Task	None

Asynchronous Continuation & Failure Boundary Semantics

Both AI inference service tasks (Llama 3.3 and Mistral Large) and the Qdrant vectorization task are configured with `camunda:asyncBefore="true"`. This isolates intensive LLM calculations onto separate background worker threads managed by the Camunda Job Executor. If transient hardware latency or token limits cause a model failure, Camunda's automated retry cycle (R3/PT10S) performs exponential backoff retries without aborting the parent process instance.

Developer Integration: Camunda 7 & 8 Handlers

Spring Boot JavaDelegate and Camunda 8 Zeebe Worker source code examples

Camunda 7: Spring Boot JavaDelegate

Package: com.personallibrary.workflow.delegate

```
@Component("llamaSummarizationDelegate")
public class LlamaSummarizationDelegate implements JavaDelegate {
    @Autowired
    private AiSummarizationService summarizationService;

    @Override
    public void execute(DelegateExecution execution) throws Exception {
        String title = (String) execution.getVariable("documentTitle");
        String content = (String) execution.getVariable("fullContent");
        BibTeXMetadata bibtex = (BibTeXMetadata)
            execution.getVariable("bibtex");

        SummaryRecord summary = summarizationService
            .generateAnalyticalSummary(title, content, bibtex);

        execution.setVariable("llamaSummary", summary);
    }
}

@Component("vectorRagDelegate")
public class VectorRagDelegate implements JavaDelegate {
    @Autowired
    private VectorRagService vectorRagService;

    @Override
    public void execute(DelegateExecution execution) throws Exception {
        String docId = (String) execution.getVariable("documentGuid");
        String text = (String) execution.getVariable("fullContent");

        List<DocumentChunk> chunks = vectorRagService.index(docId,
            text);
        execution.setVariable("vectorChunks", chunks);
    }
}
```

Camunda 8: Zeebe JobWorker

Package: com.personallibrary.workflow.worker

```
@Component
public class DocumentIngestionZeebeWorkers {
    @Autowired
    private AiSummarizationService summarizationService;
    @Autowired
    private VectorRagService vectorRagService;

    @JobWorker(type = "llama-summary-task", autoComplete = true)
    public Map<String, Object> handleLlama(final ActivatedJob job) {
        Map<String, Object> vars = job.getVariablesAsMap();
        String title = (String) vars.get("documentTitle");
        String content = (String) vars.get("fullContent");

        SummaryRecord summary = summarizationService
            .generateAnalyticalSummary(title, content, null);
        return Map.of("llamaSummary", summary);
    }

    @JobWorker(type = "qdrant-vectorization-task", autoComplete = true)
    public Map<String, Object> handleQdrant(final ActivatedJob job) {
        Map<String, Object> vars = job.getVariablesAsMap();
        String docId = (String) vars.get("documentGuid");
        String text = (String) vars.get("fullContent");

        int chunks = vectorRagService.index(docId, text).size();
        return Map.of("chunksCount", chunks, "status", "INDEXED");
    }
}
```